Music Streaming Database

Problem

The music industry has undergone a fundamental shift from physical media and downloads to on-demand streaming, yet the data infrastructure behind most streaming platforms remains fragmented and hard to scale. Artists release music across thousands of albums, songs credit multiple collaborators in different roles, and users generate billions of play events every day — all of which must be tracked accurately for royalty attribution, recommendations, and analytics. Existing solutions often store catalog data in silos, fail to capture featured artist credits properly, and cannot efficiently query listening behavior at scale.

StreamIQ is a music streaming platform that solves this by centralizing all music and user data into a single relational database. By modeling artists, albums, songs, playlists, and listen history together, the platform creates one source of truth for the entire catalog and every user interaction. Using structured relationships and derived analytics, StreamIQ can accurately attribute streams to the right artists, surface personalized recommendations based on listening history, and generate monthly listening snapshots — giving both users and rights holders a clear, real-time view of how music is consumed.

Highlighted Nouns and Verbs:

Verbs

Nouns

The music industry has shifted from physical media and downloads to on-demand streaming, but the underlying data systems that manage music catalogs and user activity often lack structured relationships and scalability. Artists **release** albums, albums **contain** songs, and songs frequently **credit** multiple artists in different roles such as featured performers or producers. At the same time, users **create** playlists, **follow** artists, and **play** songs billions of times per day. Each play event must be logged accurately to ensure proper royalty attribution, personalized recommendations, and usage analytics. However, many existing systems store catalog and listening data in isolated silos, fail to properly model song credits and many-to-many relationships, and struggle to efficiently query large-scale listening behavior.

StreamIQ is a music streaming platform that addresses these challenges by centralizing all music and user activity data into a single relational database. The system models how artists **release**

albums, how albums **contain** songs, how songs **appear in** playlists, and how users **create**, **follow**, and **listen to** content. Every time a user **plays** a song, the system **creates** a listen record with a timestamp and duration, ensuring accurate tracking of consumption. By structuring these relationships clearly, StreamIQ establishes a single source of truth for catalog management and user interaction. This allows the platform to correctly attribute streams to credited artists, generate personalized recommendations based on listening history, and produce monthly listening summaries — providing both users and rights holders with a transparent, real-time view of how music is consumed.

Problem description:

The music streaming industry must manage complex relationships between artists, albums, songs, playlists, and user activity at massive scale. Songs often credit multiple artists, users create and share playlists, and every play must be recorded accurately to support royalties, recommendations, and analytics. StreamIQ addresses this challenge by designing a centralized relational database that models these entities and their relationships in a structured, scalable way.

Task 1 — Requirements Document

Database Rules

- Each artist has a name, country of origin, and primary genre.
- An artist can release one or more albums. Each album belongs to exactly one primary artist.
- Each album has a title, release date, and type (Single, EP, or LP), and contains one or more songs.
- Each song has a title, duration, track number, and explicit content flag, and belongs to exactly one album.
- A song may credit multiple artists (e.g. featured artists, producers). Each credit has a role.
- Each user has a unique username, email, and a subscription tier (Free, Premium, or Family).f
- A user can create zero or more playlists. Each playlist belongs to exactly one user.
- A playlist has a name, visibility setting (Public or Private), and an ordered list of songs.
- A song can appear in multiple playlists; a playlist can contain multiple songs.
- Every time a user plays a song, a listen record is created with a timestamp and play duration.
- A user can follow zero or more artists.

Identified Nouns

Noun	Role in Database
User	A registered account that streams music and creates playlists
Artist	A musician or band that releases music
Album	A music release containing one or more songs
Song	An individual track that can be streamed
Playlist	A user-curated ordered list of songs
ListenHistory	A log entry recording a user playing a song
SongArtist (association)	Links songs to artists with a credited role
PlaylistSong (association)	Links songs to playlists with a position order

Identified Actions

Action / Verb	Relationship It Describes
releases	Artist releases Albums — 1 to many
contains	Album contains Songs — 1 to many

credits / performs	Song credits Artists — many to many
creates	User creates Playlists — 1 to many
appears in	Song appears in Playlist — many to many
listens to / plays	User plays Song, logged in ListenHistory
follows	User follows Artist — many to many

Redis Addition — In-Memory Functionality

For Project 3, StreamIQ introduces an in-memory key-value store (Redis) alongside the existing database. The goal is to move one high-traffic, read-heavy functionality out of persistent storage and into Redis, where it can be served with sub-millisecond latency and updated atomically without locking the primary database.

Selected Functionality: Top Songs Leaderboard

The functionality selected for Redis is the Top Songs Leaderboard — a real-time ranking of every song in the catalog by total play count. In the original relational design, generating this ranking requires aggregating millions of rows in the ListenHistory table with GROUP BY song_id and ORDER BY COUNT(*). This query is expensive, runs constantly (every user sees the leaderboard on the home page), and becomes slower as the catalog and play history grow.

Moving this use case to Redis solves three problems at once:

- **Read performance:** ZREVRANGE on a sorted set returns a pre-ranked list in O(log N + M) time, compared to a full-table aggregation in SQL.
- **Write performance:** Each new play becomes a single atomic ZINCRBY instead of an INSERT into a large ListenHistory table that must then be re-aggregated.
- **Concurrency:** Multiple users playing the same song simultaneously update the score atomically, with no row-level locking or race conditions.

Scope of Redis Implementation

The Redis layer is scoped to the leaderboard use case and its associated song metadata. Specifically, it supports the following user-facing operations:

- Viewing the ranked leaderboard of all songs by play count.
- Adding a new song to the leaderboard (with optional initial play count).
- Logging a play event for a song (incrementing its score).
- Editing a song's metadata (title, artist, album, duration).
- Removing a song from the leaderboard.
- Resetting and reseeding the leaderboard to a known baseline state.

All other entities — users, playlists, full listen history, artist follow graph — remain in the primary database and are out of scope for this project. The Redis store is treated as a performance layer for the leaderboard feature, not a full replacement for persistent storage.